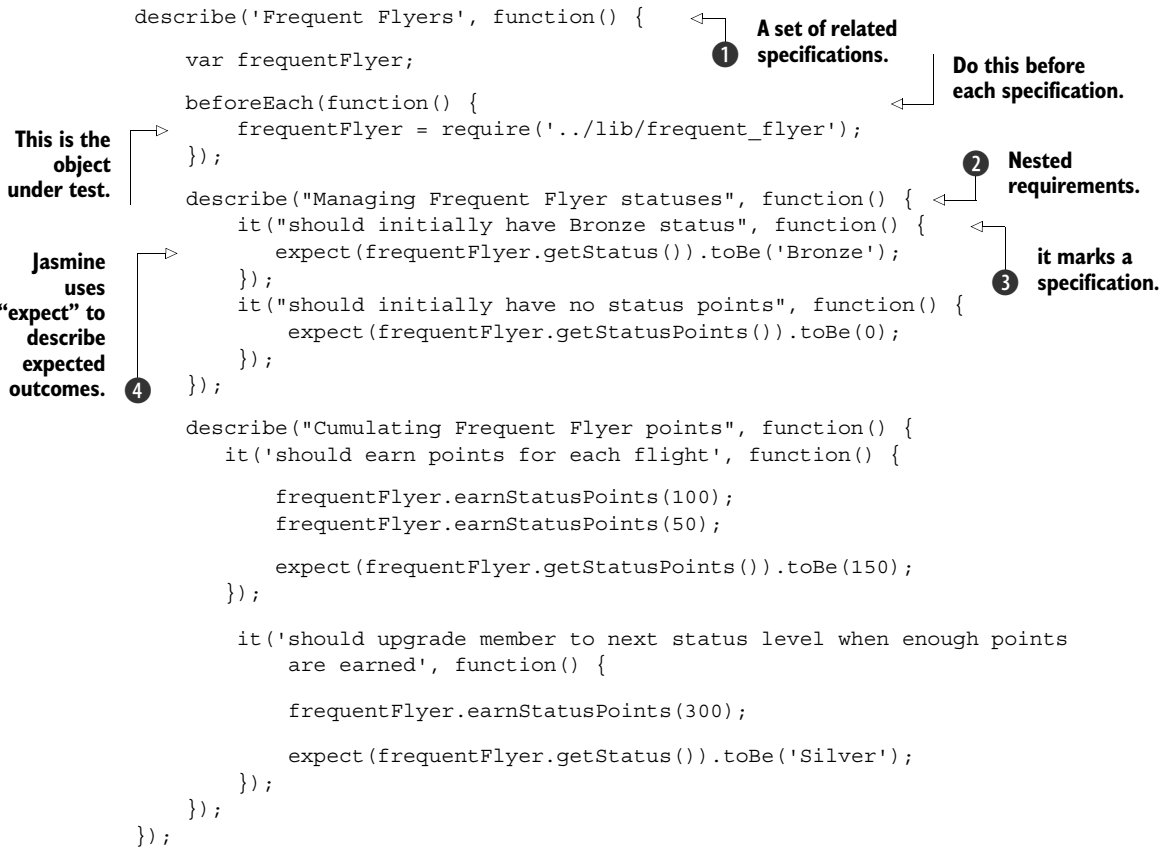
Jasmine lets you describe granular requirements in a format similar to RSpec. It supports declarative, example-based specifications, nested contexts, and its own mocking library. Here's a simple example of some requirements expressed using Jasmine:




```
describe('Frequent Flyers', function() {

    var frequentFlyer;

    beforeEach(function() {
        frequentFlyer = require('../lib/frequent_flyer');
    });

    describe("Managing Frequent Flyer statuses", function() {
        it("should initially have Bronze status", function() {
            expect(frequentFlyer.getStatus()).toBe('Bronze');
        });
        it("should initially have no status points", function() {
            expect(frequentFlyer.getStatusPoints()).toBe(0);
        });
    });

    describe("Cumulating Frequent Flyer points", function() {
       it('should earn points for each flight', function() {

           frequentFlyer.earnStatusPoints(100);
           frequentFlyer.earnStatusPoints(50);

           expect(frequentFlyer.getStatusPoints()).toBe(150);
       });

        it('should upgrade member to next status level when enough points
            are earned', function() {

            frequentFlyer.earnStatusPoints(300);

            expect(frequentFlyer.getStatus()).toBe('Silver');
        });
    });
});
```

As you can see, Jasmine specifications have a layout that's very similar to the layout of RSpec specifications. Groups of related specifications are marked by the `describe` function ❶, which can contain specifications or more nested `describe` functions ❷. Each specification is marked by the `it` method ❸, which is made up of a title and a function containing the specification test code. Jasmine uses the `expect()` function ❹ to describe expected outcomes, though as you'll see in section 10.5.2, this isn't your only option.

EXECUTABLE SPECIFICATIONS IN .NET WITH NSPEC

You've seen how you can write NUnit tests in a more BDD style. But for teams who prefer a purer BDD approach, NSpec is an elegant BDD library that lets you write RSpec-style specifications directly in .NET (http://nspec.org/).

A sample set of specifications can be seen here:

```
public class WhenUpdatingStatusPoints : nspec
{
    FrequentFlyer member;
```